BORDER OPTICS — Development Log

Entry 1: Project Framing and Motivation

What this project is. BORDER OPTICS is an independent satellite-verification study of India's Vibrant Villages Programme (VVP), the Government of India's border-area development scheme launched in 2022–23. VVP-I sanctioned roughly ₹4,800 crore (later detailed as 2,558 works worth ₹3,431 crore) across 2,967 border villages in 46 blocks, 19 districts, and 5 states/UTs — Arunachal Pradesh, Himachal Pradesh, Uttarakhand, Sikkim, and Ladakh — with 662 of those villages designated "priority" villages for Phase 1 development.

Why this project exists. VVP sits at the intersection of political science, geospatial science, and economics — three fields this project treats as inseparable rather than academic silos. Politically, VVP is a textbook case of securitization theory (Buzan and Waever): border infrastructure framed as civilian welfare development, but functioning simultaneously as a geopolitical signal along a contested frontier. Economically, it is a large, multi-year public investment whose actual delivery has never been independently measured. Geospatially, it is a rare case where a government development claim can be tested directly against satellite evidence — built-up area change, road construction, night-time lights — rather than taken on faith.

The empirical hook. A parliamentary reply on record states explicitly that "no impact assessment has been carried out" for VVP. That sentence is the reason this project exists: a scheme this large, this strategically framed, and this expensive has no independent, public, evidence-based accounting of whether the development it promised has actually happened on the ground.

Aim. To independently verify, using multi-temporal satellite imagery and open government data, whether VVP-I's "priority" villages show measurable physical development (built-up area expansion, road network growth, night-time light intensity change) between programme sanction and the present — and whether that development, if present, correlates with each state's sanctioned budget and project count, or diverges from it in ways that raise questions about implementation, prioritization, or securitization logic overriding genuine developmental need.

Research Questions. RQ1: Do VVP-I priority villages show statistically detectable built-up area growth between a pre-programme baseline and the present, using Sentinel-2/Landsat imagery? RQ2: Does the magnitude of physical change correlate with each state's sanctioned VVP budget and project count, or are there states where large budgets have not translated into detectable ground change? RQ3: Do night-time light (VIIRS) trends corroborate or contradict the built-up area findings? RQ4: Is there a spatial or political pattern to where development is concentrated — for example, villages closer to the Line of Actual Control developing faster than those further back, consistent with a securitization-driven rather than needs-driven allocation logic?

Hypotheses. H1: Priority villages will show a statistically significant increase in built-up area after programme sanction compared to a matched pre-programme baseline. H2: The size of this change will not be uniformly proportional to sanctioned budget — some high-budget states will show comparatively muted physical change, and vice versa. H3: Villages nearer the international border/LAC will show disproportionately faster change than villages further from it within the same state, consistent with securitization theory's prediction that border proximity, not developmental need, drives priority.

Scope and current status (as of this entry). The foundation of this project is a complete, verified, village-level dataset of all 662 VVP-I priority villages across the 5 states/UTs — village/habitation name, district, block, and (where available) LGD code, Census 2011 households/population, and coordinates — since satellite verification requires knowing exactly where to look before any imagery analysis can begin. As of this entry: Arunachal Pradesh (455/455 villages) is fully compiled and verified against the official state government source, including LGD codes and Census figures. Sikkim (46/46 villages) is fully compiled and verified against a Rajya Sabha parliamentary annexure. Uttarakhand (51/51 village names) is compiled and cross-verified against official block-wise totals, though the block assignment for 19 of Pithoragarh's villages remains genuinely unresolved pending a primary source pairing name to block. Himachal Pradesh (75 villages) and Ladakh (35 villages) remain incomplete — a documented, ongoing data-availability limitation rather than an unattempted one, with specific leads (an unindexed page in a Himachal government PDF; an RTI-only path for Ladakh) identified and being pursued.

This entry establishes the project's intent before the technical log below documents the actual acquisition process, sourcing decisions, and verification steps used to reach that status.

Entry 2: Village-Level Dataset Acquisition Across the Five States/UTs

With the project's framing established in Entry 1, the next task was building the foundation every downstream analysis depends on: a verified, village-level list of all 662 VVP-I priority villages across Arunachal Pradesh, Sikkim, Uttarakhand, Himachal Pradesh, and Ladakh. This turned into the most time-consuming phase of the project so far, and it did not resolve cleanly for all five.

Arunachal Pradesh — fully resolved (455/455). Sourced the complete list from the Arunachal Pradesh state government's official identified-villages document. It included district, block, habitation name, LGD code, and Census 2011 households/population for every entry. Verified it by counting rows and cross-tabulating by district — the totals matched the officially reported "455 villages across 11 districts" figure exactly, so this dataset is complete and fully sourced from a primary government document.

Sikkim — fully resolved (46/46). Found the actual village-wise annexure inside a Rajya Sabha Unstarred Question reply (Question No. 2321, dated 09 August 2023) from the Ministry of Home Affairs — this is a primary parliamentary document, not a secondary source. It lists district, block, and village name for all 46 Sikkim villages. Verified the count by tabulating district and block totals (East: 4, North: 42), which is internally consistent. LGD codes, Census figures, and coordinates are not in this source, so those fields remain blank for Sikkim and will need to be geocoded separately by name and district.

Uttarakhand — mostly resolved (51/51 names, block assignment partial). The Uttarakhand Rural Development Department's VVP page gave the official block-wise and district-wise village counts (Pithoragarh 27 — split Munsyari 8, Dharchula 17, Kanalichhina 2; Chamoli 14, all under Joshimath block; Uttarkashi 10, all under Bhatwari block) but did not publish the actual village names. The names themselves came from a secondary tourism/information source (euttaranchal.com), which listed exactly 51 villages split 14/27/10 across the three districts — an exact numeric match to the official government counts, which gives real confidence in the list despite it not being a primary government citation itself. Cross-checked the block assignment against known regional geography: Chamoli's 14 and Uttarkashi's 10 map cleanly onto Joshimath and Bhatwari respectively since the block total equals the full district total. For Pithoragarh, identified 8 of the 27 villages (Bilju, Burphu, Martoli, Milam, Pachhu Gunth, Panchhu, Rilkot, Tola) as the Munsyari block set, since these are the well-documented Johar Valley settlements. The remaining 19 Pithoragarh villages could not be confidently split between Dharchula (17) and Kanalichhina (2) from any source found — no document pairs individual village name to block for this subset, so those rows are marked "unresolved" rather than guessed. This does not block the satellite analysis itself (which only needs village name + district to geocode), only the administrative block label for those 19 rows.

Himachal Pradesh — genuine, documented gap (7 of 51 inhabited villages named). This state does not have a public village-wise annexure. What I was able to confirm: Himachal identified 703 total border villages, of which 75 were selected as VVP-I priority villages, and of those 75 only 51 are actually inhabited (32 in Kinnaur, 19 in Lahaul-Spiti) — the other 24 have no population. An Action Plan for these 51 villages, worth ₹658.31 crore, was submitted to the Ministry of Home Affairs on 11 September 2023. Checked Rajya Sabha Question No. 401 (2025) and Lok Sabha Question No. 2104 (2023) — both confirm the count of 75 but neither includes an annexure of names. News coverage across several Tribune articles named individual villages in passing while covering unrelated project launches — Chhitkul, Pooh, Nako, Leo, and Chango in Kinnaur, and Gue and Lalung in Spiti — which is how 7 of the 51 names got confirmed. Beyond that, the actual Action Plan submitted to MHA (which almost certainly contains the full village list) was never publicly indexed anywhere I could find it.

Ladakh — genuine, documented gap (0 of 35 named). Confirmed 35 revenue villages are split across Durbuk and Nyoma sub-divisions from a DC Leh review-meeting notice and matching PIB releases, but no source gives the actual names. Lok Sabha Question No. 4360 (2025) specifically asked for the list of selected villages; the publicly available reply text does not include the annexure. Checked whether a village-tourism directory (rural.tourism.gov.in) or a general village database (viewvillage.in, which lists 6 total revenue villages in Durbuk block: Chushul, Durbok, Kargyam, Man Pangong, Shachokol, Tagste) could substitute — neither confirms which villages are actually VVP-I selected versus just administratively present in the block, so neither was usable as a substitute for an actual scheme-eligibility list.

Decision on how to close this gap. Considered filing RTI applications with the DC Kinnaur/ADC Spiti office (Himachal) and DC Leh office (Ladakh), since both cases involve a specific, named, dated government Action Plan document that demonstrably exists but was never published online. Decided against filing for now — the 30-day RTI turnaround isn't worth holding up the rest of the project, and this isn't information being deliberately withheld from the public record so much as never digitized/indexed.

Final scope decision for the analysis. Rather than let this incomplete state block the project, decided to explicitly scope the village-level satellite analysis (built-up area change, VIIRS night-light trend, border-proximity pattern) to the three states/UTs with complete official village identification: Arunachal Pradesh, Sikkim, and Uttarakhand — 552 villages, a full statistical sample. Himachal Pradesh's 7 confirmed villages will be carried as a small, clearly-labeled illustrative case study, not part of the core statistical sample. Ladakh will be excluded from village-level analysis entirely and discussed only at the state aggregate level. The budget-versus-development correlation question (RQ2) still runs across all five states/UTs, since sanctioned budget and project counts are known at the state level for all five regardless of village-name completeness.

This scope restriction — and the reasoning behind it — will be stated explicitly in the Data and Methodology section of the Research Paper, not left implicit, so the coverage gap is transparent to any reader rather than something they'd have to infer from missing figures later on.

Entry 3: Geocoding Pipeline and Coordinate Resolution

With 552 villages confirmed by name across Arunachal Pradesh, Sikkim, and Uttarakhand (plus 7 named Himachal Pradesh villages carried as an illustrative case study), the next task was converting each into a precise coordinate that satellite imagery pipelines could actually query against.

First pass — OpenStreetMap (Nominatim). Built a script that queries each village by name, block, district, and state, falling back to a simplified query (dropping the block name) when the full query returned nothing. Iterated through several bugs before this was reliable: an initial version using the `geopy` library kept failing with connection timeouts that persisted even after increasing the configured timeout value — an internal quirk in how geopy resolves its timeout parameter rather than a genuine network problem. Rewrote the geocoding calls to hit Nominatim's HTTP API directly via `requests`, with an explicit per-call timeout, retry-with-backoff logic, and incremental checkpoint saving every 10 rows so an interrupted run never loses more than a few rows of progress. Also excluded "Forest Block" entries from geocoding attempts entirely — these are forest survey compartments included in the Sikkim VVP list under village jurisdiction, not actual inhabited settlements.

Second pass — Bhuvan (ISRO) Village Geocoding API. Registered for API access and used Bhuvan's Census-linked village geocoding endpoint as a fallback for everything Nominatim missed. This API returns richer data than OSM (village code, household count, population alongside coordinates), but testing revealed it does not filter by state or district — a common village name can silently return a same-named village in a completely different state. A direct test confirmed this: querying "Kharman" (an Anjaw district, Arunachal Pradesh village) returned a same-named village in Jhajjar district, Haryana, instead. Added a validation step that only accepts a Bhuvan result if its returned district name matches the village's expected district — any mismatch is discarded as a name collision rather than treated as a genuine match. A second, separate bug surfaced during this phase: a pandas column that starts out entirely empty gets inferred as a float64 column, which then rejects the (string-typed) village codes Bhuvan returns — fixed by explicitly casting those columns to object dtype before writing to them. Also had to fix a resume-logic bug where re-running the Nominatim script after the Bhuvan pass briefly overwrote already-Bhuvan-matched rows, since the Nominatim script's "already done" check didn't recognize Bhuvan's status labels — temporarily dropped Sikkim and Uttarakhand's matched counts before being caught and restored.

Results: 258 of 559 attempted villages geocoded — Arunachal Pradesh 186/455, Sikkim 31/46, Uttarakhand 34/51, Himachal Pradesh 7/7. Sikkim and Uttarakhand improved meaningfully from the Bhuvan fallback (Sikkim 14→31, Uttarakhand 26→34); Arunachal Pradesh did not gain a single additional match from Bhuvan.

A substantive finding, not just a data gap. Looking at what the unmatched Arunachal entries actually are is informative in its own right. Many are not civilian revenue villages at all — they are Border Roads Task Force camps, army staging huts, labour camps, and administrative headquarters designations ("Walong BRTF Camp," "Misai Labour Camp," "Staging Hut Krosam," "Chaglagam H.Q."). These appear in the official VVP "priority village" list because the programme targets any inhabited point along the border, however small or transient, but they have no footprint in any civilian geospatial database — not OpenStreetMap, not the Census-linked Bhuvan directory — because they were never classified as villages in the first place. This is worth carrying into the Discussion section as a real observation about what "village-level development" along a securitized frontier actually consists of on the ground, rather than treating it only as a limitation to apologize for.

Scope decision. The core satellite analysis (built-up area change, VIIRS night-light trend, border-proximity pattern) will run on the 251 villages with confirmed coordinates across Arunachal Pradesh, Sikkim, and Uttarakhand, preserving the three-state comparative structure. Himachal Pradesh's 7 confirmed villages are carried as a small, separately-labeled illustrative case study. The remaining villages stay in the administrative dataset for context but are excluded from point-based satellite sampling, with the reason — genuine absence from every available public geolocation source — stated explicitly.

Entry 4: Satellite Imagery Pipeline — Google Earth Engine Setup

With 258 villages geocoded (251 in the core three-state sample, 7 in the Himachal Pradesh illustrative case study), the next task was pulling actual satellite evidence for each point — the whole reason the geocoding phase existed in the first place.

Merging into a single master table. Wrote a script to combine the four per-state geocoded CSVs into one table, keeping only villages with a confirmed match (discarding the "NO MATCH" and "EXCLUDED — forest block" rows), and adding an `is_core_sample` flag so Himachal Pradesh could be carried separately from the three-state statistical sample without needing a different pipeline. This produced `border_optics_master_villages.csv` — 258 rows, ready for upload.

Uploading to Earth Engine. Uploaded the master CSV as a Table asset through the Code Editor's Assets panel, same workflow used for Stolen Strata. The asset ID that Earth Engine assigns on ingest is not the same as the internal task ID shown mid-upload — pasted the wrong one into the script the first time (the ingest task's short internal hash instead of the full `projects/earthengine-legacy/assets/...` path), which produced a "Collection asset not found" error until corrected by copying the ID directly from the Assets panel rather than from the task log.

Building the extraction script. Wrote a script to, for each village point: buffer it by 500m (small enough to stay village-specific, large enough to average out single-pixel noise), pull a cloud-masked Sentinel-2 composite for a "before" and an "after" window, compute NDBI (Normalized Difference Built-up Index, using the SWIR1 and NIR bands) as the built-up proxy, and pull a VIIRS night-lights mean for the same two windows as an independent proxy. Hit one structural bug getting this to run: passing a buffered Feature directly into `reduceRegion`'s `geometry` argument fails, because Earth Engine's buffer operation on a Feature returns another Feature, not a Geometry, and `reduceRegion` requires a Geometry specifically — fixed by explicitly calling `.geometry()` on the buffered feature before passing it through.

First result set — full calendar year (2021 vs 2025). With the pipeline working, ran it using full-year composites (Jan-Dec) for the "before" (2021, pre VVP-I sanction) and "after" (2025, most recent complete year) windows. Result: no evidence of a significant increase in built-up area (Wilcoxon signed-rank test on paired NDBI values, $p = 1.000$ for the alternative that "after" exceeds "before") — if anything, the mean NDBI change across the 251-village core sample was slightly negative. VIIRS night-lights showed a borderline result ($p = 0.050$).

Before treating this as a finding, one thing needed checking first: a full calendar-year composite in high-altitude Himalayan terrain will include months of snow cover, and if the ratio of snow-covered to snow-free images differs between the 2021 and 2025 composites — which is plausible, since it depends on which specific days happened to be cloud-free in each year — that alone could shift the NDBI value regardless of any real change on the ground. Stolen Strata's own methodology used season-matched (Jun-Sep) composites for exactly this reason, so the same discipline needed to apply here before this result could be trusted.

Entry 5: Seasonal Compositing and a Robustness-Check Finding

Switching to season-matched composites. Changed both windows to June-September only (summer, largely snow-free across the sample's elevation range) to remove the snow-cover confound identified in Entry 4. This immediately broke the script in a new way: several villages had zero cloud-free Sentinel-2 images available in that narrower four-month window, producing an empty composite image with no bands, which crashed the NDBI computation (`normalizedDifference` on an image with no bands). Fixed by checking each village's image count for both windows before compositing, and writing a null value (rather than crashing) for any village with zero usable images in a given window — also kept the image counts themselves as output columns, since they're a direct signal of how trustworthy each village's composite actually is.

Sikkim's data disappeared entirely. Once the summer-only version ran clean, it became clear why the full-year version had looked usable in the first place: in the June-September window specifically, all 31 of Sikkim's geocoded villages returned zero cloud-free images for at least one of the two periods, wiping out Sikkim's NDBI data completely. This makes physical sense — June-September is peak monsoon in the Eastern Himalaya, so Sikkim in particular gets almost no clear satellite days in exactly the window chosen to avoid snow. The full-year composite had been masking this by allowing the algorithm to reach for whatever clear days existed at any point across twelve months; narrowing to summer-only fixed the snow problem but broke Sikkim on cloud cover instead. This is a real, geography-driven trade-off between the two seasonal choices, not a bug to code around.

The result flipped. With the summer-only composite (now n=154/251 villages with valid data after the Sikkim dropout and a smaller number of Arunachal/ Uttarakhand gaps), the same Wilcoxon test on NDBI now showed a highly significant increase ($p < 0.000001$) — the opposite conclusion from the full-year version. VIIRS night-lights, tested the same way on the same summer window, showed the opposite pattern again: $p = 0.9999$, essentially zero evidence of a positive change, actually trending slightly negative at the median.

What this means. Two composite choices, both individually defensible, produced opposite conclusions from the same NDBI-based approach — which means the underlying signal is not robust to a methodological choice that should, in principle, be a minor detail. VIIRS night-lights, the more temporally stable of the two proxies because it isn't affected by vegetation/snow phenology the way a spectral index is, gave a consistent answer across both windows: no significant evidence of increased night-time activity following VVP-I sanction, in either version of the test. Taken together, the more trustworthy signal is the one that didn't change when the methodology changed — and that signal shows no confirmed increase. Rather than picking whichever NDBI result is more convenient, the plan is to report both composite results side by side in the Research Paper as an explicit robustness check, with the instability itself treated as a finding: a single-date spectral index comparison is not sufficient to make a confident claim about built-up change in this terrain, and the more stable proxy available (night-lights) does not support a measurable development signal either way. This lines up directly with the project's founding premise — the parliamentary record stating no independent impact assessment has ever been carried out for VVP.

A second, independent observation from RQ2. Restricting to villages with valid summer-window NDBI data left only two states with enough coverage to compare: Arunachal Pradesh (120 villages, mean NDBI change +0.0284) and Uttarakhand (34 villages, mean NDBI change +0.0293) — Sikkim dropped out entirely. These two states' sanctioned VVP-I budgets differ by roughly a factor of ten (₹2,749.74 crore and 2,082 projects for Arunachal Pradesh, versus ₹270.58 crore and 200 projects for Uttarakhand), yet their average measured built-up change is nearly identical. Two data points cannot support a statistical claim, but descriptively this is consistent with H2 — budget scale is not translating proportionally into a correspondingly larger physical development signal.

Next step. With both the full-year and summer-matched results now on hand, the remaining satellite work is: deciding on the final reporting format for this robustness check in the Research Paper's Results section, and picking up RQ4 (border-proximity pattern), which still needs a Line of Actual Control / border geometry layer that hasn't been sourced yet.

Entry 6: Border-Proximity Analysis (H3)

With the built-up and night-light change values in hand for both composite windows, the last untested hypothesis was H3 — that villages nearer the international border/LAC would show disproportionately more change than villages further back within the same state, consistent with securitization theory's prediction that border proximity, not developmental need, drives where VVP resources actually land.

Sourcing a border line. Used Natural Earth's Admin 0 Boundary Lines dataset (land boundaries, 1:10m scale) as the border geometry — a standard, citable, publicly available cartographic source. This needs an explicit caveat for the Research Paper: India's international boundary with China, commonly referred to in this context as the LAC, is disputed and has no single internationally agreed alignment. Natural Earth's rendering is a cartographic simplification, not a legal or official claim, and any distance-to-border figure computed from it should be read as approximate and relative (useful for comparing villages against each other) rather than as an authoritative statement of where the boundary actually sits on the ground. The download link on the Natural Earth site had changed since it was last referenced — the boundary-lines-only URL 404s now; the current page is `10m-admin-0-boundary-lines` (previously indexed with a "-land" suffix that no longer resolves).

Computing distance per village. Loaded the 258 geocoded village points and the boundary shapefile into GeoPandas, filtered the boundary file down to the 27 segments involving India (matched on the `ADM0_LEFT`/`ADM0_RIGHT` attribute fields, since Natural Earth's boundary-line schema identifies each segment by the two countries it separates rather than by a single country code), reprojected both layers to a metric CRS appropriate for the Himalayan region (UTM 44N), and computed the straight-line distance from each village to the nearest point on the combined India border geometry. Distances ranged from 0.1 km to 69.4 km, with a mean of 27.3 km — a plausible spread for a sample explicitly selected as "priority" border villages.

Testing H3 against both composite windows. Ran a Spearman correlation between distance-to-border and each village's NDBI change and night-light change, separately for the full-year and summer-matched datasets, following the same robustness-check logic established in Entry 5 rather than trusting a single result.

Results:

- NDBI change vs. distance: not significant in either window (full-year rho = 0.043, p = 0.497; summer-matched rho = 0.037, p = 0.650). This is a stable non-finding — both composite choices agree that built-up change, as measured here, shows no relationship with border proximity.
- Night-light change vs. distance: significant in the full-year composite (rho = -0.259, p < 0.0001 — negative rho meaning villages closer to the border show more light increase, consistent with H3), but not significant in the summer-matched composite (rho = -0.076, p = 0.233).

Interpretation. The NDBI result is consistent and trustworthy precisely because it doesn't change with the compositing choice — there is no confirmed relationship between border proximity and built-up area change in this dataset. The night-light result is more complicated: it shows a strong, clean signal in one version of the analysis and no signal at all in the other, which is the same instability pattern already documented for RQ1 in Entry 5. Given that pattern, this result cannot be reported as "H3 confirmed" — the honest characterization is that there is a suggestive, non-robust signal in one methodological version, not a finding that survives the same robustness check the rest of the project's satellite results are being held to. This reinforces rather than contradicts Entry 5's broader conclusion: single-date/single-window spectral and radiance comparisons in this terrain are not stable enough, on their own, to support confident directional claims, and every result in this project needs to be reported with that caveat attached rather than cherry-picking whichever version looks cleanest.

Status after this entry. All four research questions have now been run against the satellite pipeline at least once, with RQ1 and H3 both showing the same non-robust, composite-window-sensitive pattern, and RQ2's budget-independence observation (Entry 5) still standing as the most consistent descriptive finding so far. What remains before the Research Paper stage is: deciding on final reporting language for this robustness-check framing, and building the plots/maps that will actually visualize all of this once the analysis phase is fully closed out.

Entry 7: Visualization — Charts and Interactive Maps

With all four research questions tested (Entries 5-6), the next task was turning the numeric results into visuals — three static charts and two interactive maps.

Static charts. Built three figures in Python/Matplotlib: (1) the NDBI change distribution for both composite windows side by side, visually confirming the sign-flip documented in Entry 5; (2) state-level mean NDBI change plotted against sanctioned VVP-I budget, visually confirming the budget-independence pattern (Arunachal Pradesh's much larger budget bar paired with a negative change bar, against Uttarakhand's small budget bar paired with the largest positive change); (3) the H3 border-distance scatter plots for both windows, showing the same flat/non-robust pattern identified statistically in Entry 6.

Interactive maps. Built Folium-based interactive maps of all 251 core villages, color-coded by NDBI change (blue = decrease, red = increase), with click-through popups showing each village's NDBI change, night-light change, and distance to border. Hit two design bugs building this: first, Folium's colorbar legend isn't tied to a layer's visibility toggle — it's a standalone map element — so a single combined map with both composite windows as togglable layers always showed both legends stacked on top of each other regardless of which layer was actually checked. Fixed by generating two separate map files (one per composite window) instead of forcing both into one togglable map, which also keeps each map visually cleaner on its own. Second, the initial zoom level defaulted to a centroid-based view that, given how geographically spread the three-state sample is, opened zoomed out to roughly all of South Asia rather than the actual village cluster — fixed with `fit_bounds()` so each map opens already framed on its own data.

Status. Analysis and visualization phase is now functionally complete: village data acquisition, geocoding, satellite extraction (both composite windows), all four RQ/hypothesis tests, and both static and interactive visual outputs. What remains is deciding the next build priority — a full dashboard (as with the prior project) versus moving straight into Research Paper drafting now that the plots and maps exist to draw on.

Entry 8: Dashboard, Documentation, and Publication Prep

With the analysis and visualization phase closed out, the final build priority was a multi-page Streamlit dashboard, mirroring the structure used for the prior project rather than a static write-up alone — the raw figures, interactive maps, and live-recalculating statistical tests all benefit from an explorable interface more than a fixed document does.

Dashboard structure. Built `app.py` as the entry point (Home page) with a `pages/` directory holding seven sub-pages — Study Design, Built-Up Change, Night-Lights, Statistical Validation, Explore Trends, Interactive Maps, and Methodology & Limitations — plus a shared `utils/theme.py` (watermelon-and-mint dark theme, consistent styling across every page) and `utils/data.py` (a single cached `load_data()` used by every page, merging the master village table with both compositing-window result files so `latitude`/`longitude`/`distance_to_border_km` are available wherever needed without re-reading the merge logic per page).

Live recomputation over static numbers. Several pages compute their statistics live from the underlying CSVs on every load (Wilcoxon tests on Statistical Validation, Spearman correlations on H3, state-level aggregates on Explore Trends) rather than hardcoding the numbers already reported in the Research Paper — this keeps the dashboard honest against the underlying data if it's ever regenerated, at the cost of needing the raw `data/processed/` CSVs to ship with the repository rather than only the derived figures.

Full Project Documentation section. Added a section to the Home page with three download buttons for the Research Paper, Project Journal, and Development Log as PDFs, matching the pattern used for the prior project's dashboard.

Research Paper. Drafted the formal academic write-up from the completed analysis — literature review grounding the securitization-theory framing, full methodology section documenting every acquisition, geocoding, and compositing-window decision made in Entries 1–7, results section reporting both compositing windows side by side rather than a single preferred version, and a limitations section carrying forward every documented data gap (Ladakh exclusion, Himachal illustrative-only status, Pithoragarh block ambiguity, LAC cartographic caveat) rather than treating them as resolved.

Entry 9: Panel-Readiness Review and Robustness Pass

Before treating the project as submission-ready, went back through every file — all three documents, `requirements.txt`, `app.py`, every dashboard page, and every script in `src/` — looking specifically for the kind of issue that survives a first pass: stale numbers that drifted from the data, scripts referenced in the documentation but missing from the repository, and anything that would silently fail for a reader trying to reproduce the pipeline end to end.

Village-count discrepancy. The Project Journal's Study Area section stated "262 geocoded villages," while the Research Paper, README, and the underlying `border_optics_master_villages.csv` all agree on 258. Traced this to a stale figure left over from an earlier point in the geocoding pipeline (Entry 3 records the geocoding pass evolving in stages) that never got updated in the Journal after the final Bhuvan-fallback numbers landed. Corrected to 258 throughout.

Figure numbering out of sequence. The Research Paper's Results section referenced figures in the order Figure 1, 4, 6, 5, 2, 3 — each individual reference was internally correct (pointing at the right image), but the numbering itself wasn't sequential with reading order, since figures were numbered by the order their source PNGs were generated (Entries 5 and 7) rather than the order they appear in the paper. Renumbered to a clean 1–6 sequence matching reading order; no image files needed to change, only the figure labels and captions referencing them.

A live secret committed to source, not just to `.env`. Unlike the Sentinel Hub credentials pattern from the prior project (kept in a `.env` file that `.gitignore` failed to exclude), this project's exposed credential was worse in one respect: the Bhuvan API token was hardcoded directly inside `geocode_villages_bhuvan.py` itself, committed to source control with no `.gitignore` entry that could have caught it. Moved it to an environment variable (`BHUVAN_TOKEN`, loaded via `python-dotenv`), added a `.env.example` template, and added `.env` plus common service-account-key filename patterns to `.gitignore`. Since this token was live in a public repository, it should be treated as compromised and regenerated from the Bhuvan API portal rather than reused — the old value needs to be scrubbed from git history separately, since removing it from the current file alone leaves it recoverable from any prior commit.

Missing satellite-extraction script. The single script that actually pulls Sentinel-2 NDBI and VIIRS night-lights values from Google Earth Engine — described in detail in Entry 4 (buffer-then-`.geometry()` fix, QA60 cloud masking, per-village null-safe extraction) — was never checked into `src/acquisition/`, even though every downstream script depends on its output. Reconstructed it as `extract_satellite_data.py`, matching the documented logic exactly, runnable against either compositing window via a `--window` flag, so the pipeline is reproducible from source rather than only from its cached CSV outputs.

Missing full-year counterpart to the core analysis script. `analyze_results.py` only ever covered the summer-matched window; the full-year analyzed output (`border_optics_village_results_analyzed.csv`, used throughout the Research Paper and dashboard) had no corresponding script in the repository that could regenerate it. Added `analyze_results_fullyear.py`, mirroring the same logic against the full-year extraction output.

`requirements.txt` incompleteness. The file listed five packages (`streamlit`, `pandas`, `numpy`, `scipy`, `plotly`) against a pipeline that actually imports `geopandas`, `shapely`, `matplotlib`, `requests`, `folium`, `branca`, and (once the satellite-extraction script above was restored) `earthengine-api` and `python-dotenv`. A clean install from this file alone would have failed the moment any acquisition or analysis script ran. Expanded it to match actual usage.

Full Project Documentation download buttons were dead. The three download buttons on the dashboard's Home page pointed at `Research_Paper.pdf`, `Project_Journal.pdf`, and `Development_Log.pdf` — none of which had ever been generated; only the three source `.md` files existed. Every visit to the Home page was silently showing "file not found" warnings in place of working downloads. Built all three PDFs from the source Markdown (via `pandoc`/`wkhtmltopdf`, embedding the actual result figures) and added `build_docs_pdfs.sh` so they can be regenerated whenever the underlying `.md` files change, rather than going stale again.

Stale output path in a superseded fix script. `fix_lights_map.py` (an earlier patch script addressing the night-lights map's colorbar/marker color issue, later folded properly into `make_interactive_map.py`) still saved its output to `outputs/maps/`, a path that was never correct relative to the rest of the pipeline's `outputs/interactive_maps/maps/` convention. Corrected the path for consistency, though `make_interactive_map.py` is the version actually used to regenerate the live maps.